

Verifier-Guided Prompting for Intent-to-Configuration Translation: A Preregistered NetOps Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory trial to test whether constrained prompting plus a deterministic verifier-guided generate→verify→repair loop (one allowed repair) increases deterministic-verifier semantic-correctness when translating operator natural-language intents into low-level device configurations. Design: N=150 synthetic intents sampled from a deterministic generator, shared_initial_candidate generation semantics, model = gpt-5-mini, deterministic validator = deterministic_netops_validator_v5, one initial generation per intent shared across baseline and guarded conditions, guarded pipeline permitted ≤1 repair call. Results (artifact-grounded): baseline = 149/150 (99.333%); guarded = 150/150 (100.0%); paired absolute difference = 0.006667 (≈0.67 percentage points); exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.00, 0.02] (10,000 resamples, seed = 1729). Provider accounting: completed episodes = 300; model_calls_used = 151; repair-stage invocations = 1. Interpretation: on this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow produced a numerically negligible and statistically non-significant improvement over a near-ceiling baseline. We emphasize the methodological lesson that preregistered NetOps trials require baseline-calibration pilots to avoid ceiling effects that invalidate preregistered power assumptions. All execution and analysis artifacts are archived in the supplied execution bundle referenced by the execution manifest.

I. INTRODUCTION

Natural-language intent interfaces aim to reduce operator burden by letting humans express desired network changes as free text that is automatically translated into executable device configurations. Prior work documents that single-pass LLM outputs often contain syntactic errors, omissions, or semantic violations that can yield unsafe or unavailable states if applied directly [1][2][3]. A commonly proposed defense is a generate→verify→repair architecture in which an LLM produces a candidate, a deterministic verifier checks intent-level invariants or formalized constraints, and an automated repair (or human gate) corrects violations before execution [4][5]. Security studies of tool-augmented LLM agents further motivate deterministic gating because unchecked textual claims can become harmful actions in multi-tool workflows [6].

Research question. After an autonomous literature review and preregistration we posed a confined confirmatory question: does constrained prompting (structured templates with explicit semantic slots) combined with deterministic verifier-guided iterative feedback materially increase verifier-level semantic correctness of NL→configuration translation versus

an unconstrained baseline under a paired design? The trial (study_cand_2026_b2_v1) was preregistered and frozen before any model outputs were observed.

Contributions. (1) A fully preregistered, artifactized paired confirmatory trial measuring deterministic-verifier pass/fail on a programmatically generated synthetic corpus (N=150 paired intents). (2) Artifact-backed outcomes showing that, under shared_initial_candidate semantics and a one-repair budget, the guarded pipeline raised verifier pass rate from 149/150 to 150/150 (+0.67 pp), a numerically negligible and statistically non-significant improvement (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). (3) A methodological recommendation that preregistered NetOps trials include baseline-calibration pilots and explicitly specify generation semantics (shared_initial_candidate vs independent sampling) because semantics materially change the estimand and interpretation.

II. RELATED WORK

Related literature falls into three strands; each strand provides partial motivation for the guarded generate→verify→repair architecture we evaluate and clarifies where prior claims and limitations remain.

Intent-to-configuration translation. A growing body of work explores converting operator natural-language intents into formal specifications or concrete device configurations. Empirical evaluations and surveys report that single-pass translation often attains only modest semantic-correctness on curated benchmarks, motivating post-generation checks and constrained prompting to improve fidelity [1][2]. Representative studies highlight varied task formulations (intent grammars, template-based translation, and intent-slot extraction) and report that success rates depend strongly on task granularity, the expressiveness of the target formalism, and prompt design choices [1]. Systematic reviews and targeted evaluations underscore that many NL→formal pipelines still produce semantic errors that formal verifiers can detect but that the end-to-end reliability required for unattended NetOps remains an open problem [2]. These findings justify using deterministic verifiers as the primary confirmatory endpoint and motivate task-bank designs that capture common operational idioms (reachability, ACL-like constraints, VLAN/MTU sequences, and uplink switchover patterns) rather than only isolated syntactic transforms.

Generate→verify→repair architectures and verifier technology. Several recent method papers and systems integrate deterministic or policy-guided verifiers with generation to improve correctness for infrastructure-as-code and configuration-repair tasks [4][7]. Approaches range from using verifier feedback for fine-tuning to runtime repair loops that iteratively correct a candidate until invariants hold. Reported improvements are heterogeneous: gains are larger when baseline model competence is modest, when larger repair budgets or multiple independent samples are allowed, or when verifier feedback provides actionable, fine-grained corrections rather than coarse pass/fail signals [4][7]. Critically, the effectiveness of verifier-guided repair depends on verifier coverage and the fidelity of the mapping layer that translates natural-language intents to verifier inputs; prior work emphasizes rigorous unit-testing of mapping code and staged integration with formal tools [5].

A complementary thread documents mature formal verification tools (e.g., NetKAT variants, KATch, Hoyan, and other symbolic verifiers) that can act as deterministic oracles for reachability, isolation, and policy checks in controlled topologies [5][9]. These tools vary in supported abstractions, latency, and scale: some are optimized for design-time proofs while others (e.g., recent NetKAT provers and scalable overlay checkers) demonstrate faster runtime checks for smaller topologies. Prior studies therefore treat verifier selection and verifier-coverage testing as design decisions that directly affect measured semantic-correctness and external validity.

Security, prompt-injection, and claim-to-action risks. Security- and attack-oriented analyses show that LLM-driven multi-tool workflows can convert false textual claims into concrete tool calls and harmful actions unless guarded by auditable gates or hardened planning primitives [6][8]. Empirical prompt-injection studies demonstrate practical exploitation strategies and optimization-based attacks against LLM-as-judge setups; corresponding defenses (prompt hardening, verifier-aware gating, and call-auditing) reduce risk but impose trade-offs with automation throughput [6][8]. These results motivate conservative guarded architectures that retain verifier traces and normalized configurations for operator review, a design choice we make explicit when framing operational interpretation beyond raw pass/fail rates.

How this study connects and differs. The present trial operationalizes a narrowly specified guarded architecture informed by the above strands: we evaluate verifier-level pass/fail on a deterministic synthetic corpus (so verifier selection and mapping tests are central), we limit repair budget to a single repair call to match a constrained operational budget, and we adopt `shared_initial_candidate` semantics to isolate the marginal effect of repair applied to an identical first-pass candidate. These design choices intentionally expose how sampling semantics, verifier coverage, and repair budgets interact with baseline competence—issues emphasized across the cited literature—and explain why previously reported larger guarded-vs-baseline gains can co-exist with a near-ceiling outcome in tightly controlled, verifier-capable synthetic settings [4][5][7].

III. METHODOLOGY

Preregistration and confirmatory design. The study was preregistered as `study_cand_2026_b2_v1` with a frozen analysis plan before any model outputs were observed. The primary estimand was the `paired_success_rate_difference_guarded_minus_baseline` evaluated with an exact two-sided McNemar test. The confirmatory sample specified $N=150$ distinct synthetic intents, inclusion/exclusion rules (deterministic ambiguity detector; verifier-coverage checks), a deterministic retry policy for provider timeouts (one retry), and a target power for detecting an absolute +15 percentage-point improvement ($\approx 80\%$ power) under conservative discordant-pair assumptions. The preregistration and execution contract are archived and referenced in the execution manifest.

Execution semantics: `shared_initial_candidate` and budgeted repair. The execution adapter (`hosted_netops_gvr_v1`) enforced `shared_initial_candidate` semantics: for each intent exactly one initial model generation was produced and that identical sampled candidate was used to evaluate both baseline and guarded conditions. The guarded pipeline could invoke at most one additional repair call only if the deterministic verifier failed that initial candidate. Under these semantics the baseline rate measures the validity of the single initial sampled candidate; the guarded vs baseline contrast therefore isolates the marginal effect of the single allowed repair applied to the identical candidate. This sentence is included here in Methods (per reviewer request) to prevent misinterpretation that different first-pass generations were drawn per condition. By design the adapter recorded one fresh API session per model call and audited provider calls.

Model, sampling, and deterministic validator. Declared model: `gpt-5-mini` (provider record: `openai_responses`). The archived `model_configuration.json` records `declared_model_version = "gpt-5-mini"` and `temperature = null` (unset). Because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions; we state this explicitly here (and repeat in the Disclosure Statement) to comply with preregistration/runtime-parameter reporting rules. Deterministic validator: `deterministic_netops_validator_v5` (`validator_version = 5.0`) applied a `full_intent_constraint_validation` scoring rule and returned a boolean pass/fail plus normalized final-configuration text and diagnostics (`parsed_command_count`, `required_command_count`, `violation_codes`).

Task generator, corpus, and prechecks. Confirmatory tasks were produced deterministically by the preregistered generator (`deterministic_netops_task_generator_v5`) and span four intent families: reachability/isolation, ACL-like access changes, VLAN/MTU sequence changes, and uplink switchover patterns. Topologies were restricted to verifier-capable small networks (4–32 nodes). Pre-execution mapping and verifier-coverage unit-tests were run; no confirmatory exclusions occurred. The generator produced tasks with varied declared

difficulty labels (medium, hard, very_hard) and explicit required_sequence constraints; representative tasks and their generator seeds are archived.

Execution accounting and contamination controls. Planned episodes = 300 (150 tasks $\times$ 2 conditions); completed_episode_count = 300; complete_pair_count = 150. Provider-call accounting shows model_calls_used = 151 (150 shared_initial generations + 1 repair call). The provider audit reports completed_hosted_model_call_event_count = 151, failed_hosted_model_call_event_count = 0, cache_miss_count = 151, and stage_counts = {shared_initial_generation: 150, repair: 1}. Contamination checks and mapping unit-tests passed pre-execution and contamination_summary.csv recorded zero flags. All per-call runtime metadata, mapping inputs, and verifier logs were archived for reproducibility verification.

Scoring and statistical analysis. Primary outcome: deterministic verifier pass/fail (binary). The confirmatory analysis used an exact two-sided McNemar test on the paired contingency counts and computed a paired bootstrap percentile 95% confidence interval (10,000 resamples; bootstrap_seed = 1729). The preregistered treatment of incomplete pairs for the primary test was exclusion (complete_pair_primary). Secondary contrasts and multiplicity corrections were preregistered but, under shared_initial_candidate semantics and the enforced adapter constraints, these contrasts were limited; all analysis artifacts and code are archived.

IV. RESULTS

Execution and provider audit. The run completed as planned: completed_episode_count = 300 and complete_pair_count = 150. Provider-call accounting (audited at the provider-call level) shows model_calls_used = 151; completed_hosted_model_call_event_count = 151; failed_hosted_model_call_event_count = 0; cache_miss_count = 151. Stage-level counts recorded by the execution adapter were: shared_initial_generation = 150 and repair = 1. Contamination and mapping checks reported zero flags and no pre-execution unit-test failures.

Primary numerical outcomes and paired contingency. The paired 2×2 contingency (guarded $\times$ baseline) reported by the deterministic analysis executor is: $n_{11} = 149$ (both-pass), $n_{10} = 1$ (guarded-only-pass), $n_{01} = 0$ (baseline-only-pass), $n_{00} = 0$ (both-fail). Per-condition totals: baseline_success_count = $149/150 = 0.9933333333333333$ and guarded_success_count = $150/150 = 1.0$. The paired absolute difference (guarded - baseline) = 0.006666666666666671 (≈ 0.67 percentage points). These counts are the basis for the confirmatory tests and are reproduced verbatim from the archived paired contingency artifact.

Statistical tests and bootstrap inference. The preregistered exact two-sided McNemar test on the observed discordant pair counts ($n_{10} = 1$, $n_{01} = 0$) returned $p = 1.0$ under the exact test implementation used by the paired_binary_analysis_v1 executor. Paired bootstrap percentile 95% confidence intervals were computed with bootstrap_resamples = 10000 and bootstrap_seed = 1729; the 95% CI for the paired absolute

difference is [0.00, 0.02]. These analysis parameters and results are recorded in the archived analysis artifacts and analysis log and are reported here without modification.

Repair diagnostics and revision iterations. Consistent with the near-ceiling initial success rate, only a single guarded repair invocation occurred (1 of 150 guarded episodes). That single repair converted one initially failing candidate into a passing configuration under the deterministic validator; all other guarded episodes accepted the shared initial candidate without invoking repair. Summary statistics for revision iterations in guarded episodes: median = 0, IQR = 0. Validator traces for each episode include both validation_before and validation_after snapshots, normalized_configuration strings, parsed_command_count, required_command_count, observed_touched_field_count, and any violation_codes; these traces substantiate the single repair event and the distributional summary above.

Representative per-task diagnostics (artifact-grounded). Representative validator diagnostics for tasks 000001–000006 (selected from the archived representative set) illustrate variation in declared difficulty and concrete parsing outcomes while confirming consistent validator behavior: - task-000001 (declared difficulty: medium): parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, valid = true. - task-000002 (declared difficulty: hard): parsed_command_count = 2, required_command_count = 2, observed_touched_field_count = 2, valid = true. - task-000003 (declared difficulty: hard): parsed_command_count = 6, required_command_count = 6, observed_touched_field_count = 4, valid = true. - task-000004 (declared difficulty: very_hard): parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, valid = true. - task-000005 (declared difficulty: very_hard): parsed_command_count = 3, required_command_count = 3, observed_touched_field_count = 3, valid = true. - task-000006 (declared difficulty: very_hard): parsed_command_count = 6, required_command_count = 6, observed_touched_field_count = 4, valid = true. These per-task metrics and 'valid' flags are drawn directly from archived validator traces for the representative tasks and show that the verifier checked field-level counts, required sequences, and produced normalized configurations used for scoring.

Reproducibility parameters and analysis artifacts. Key analysis seeds and parameters used in inference are published in the artifacts: bootstrap_seed = 1729 and bootstrap_resamples = 10000. The complete paired contingency table, condition summaries, contamination summary (empty), and the full set of validator traces and raw model responses are archived. Provider accounting (model_calls_used = 151; repair invocations = 1) and execution manifest entries are recorded as part of the archived execution bundle to allow direct re-computation of the exact reported analyses.

Missingness, exclusions, and sensitivity checks. The preregistered missingness and exclusion rules were not triggered: failed_hosted_model_call_event_count = 0 and unscorable

episodes = 0; no ambiguity-detection exclusions, parser failures, or verifier-unsupported-feature exclusions occurred. Consequently the primary analysis used `complete_pair_primary` and did not require sensitivity treatments; the alternate preregistered sensitivity (treating missing guarded outputs as failures) was not invoked because there were no missing or failed calls. The analysis artifacts include `missingness_summary` and `contamination_summary` CSVs confirming these counts.

Operational interpretation of the numeric outcome. The observed baseline success rate ($\approx 99.33\%$) leaves essentially no headroom for the preregistered detectable absolute improvement (target = +15 pp), producing a classical ceiling effect relative to the preregistered power assumptions. Under the `shared_initial_candidate` semantics used here the baseline measures the validity of the single sampled initial candidate and the guarded contrast isolates the marginal value of at most one repair applied to that identical candidate; hence, a near-perfect initial sample reduces the potential marginal utility of a single-repair guarded pipeline. The single observed repair shows the guarded pipeline can correct rare initial failures, but the aggregate effect on the preregistered estimand is numerically negligible and statistically unsupported (McNemar $p = 1.0$, bootstrap CI includes zero). These numeric conclusions are strictly artifact-grounded and follow directly from the archived contingency counts and analysis parameters.

Contextual diagnostics for practitioners. Beyond the binary pass/fail metric, the archived validator traces demonstrate additional operational artifacts that a guarded system would produce in deployment: normalized configuration text suitable for human audit, `parsed_command_count` and `required_command_count` to quantify task complexity, and `violation_codes` that explain specific invariant violations. Even when a marginal increase in pass rate is small on a given corpus, these deterministic diagnostics are operationally useful for triage, human review, and auditable gating; the execution bundle contains representative validator traces illustrating these affordances.

V. DISCUSSION

Ceiling effect and implications for preregistration. The preregistered power calculation assumed a substantially lower baseline; observed `baseline_success_rate` = 0.9933 left almost no headroom for the guarded repair to show a large absolute improvement. This ceiling effect invalidated the preregistered detectable-effect assumptions (target = 15 percentage points) and reduced the trial’s ability to demonstrate benefit. Concretely, with $n_{11} = 149$ and only a single discordant pair ($n_{10} = 1$), the paired analysis is dominated by an almost-complete concordance that leaves little information for estimating discordant behavior. We therefore recommend that preregistered NetOps trials include a short baseline-calibration pilot (for example, 20–40 tasks sampled from the planned generator and prompt style) to estimate baseline competence and either increase planned N or raise task difficulty before committing to confirmatory sampling. The archived deterministic task generator seeds and representative tasks permit such

a pilot: one can sample the same generator with a small pilot, measure `baseline_success_rate`, and then choose confirmatory N or difficulty stratification so that the expected baseline leaves statistical headroom for the target absolute improvement.

Why the synthetic corpus produced a high baseline. The deterministic generator produced tasks with declared difficulty markers (medium, hard, very_hard) and explicit required_sequence constraints; inspection of representative traces (tasks 000001–000006) shows many tasks have fully specified required sequences and constrained action spaces (`allowed_fields`, `allowed_touched_fields`) that reduce translation ambiguity. When intents are tightly specified and the verifier coverage matches the required invariants, even a single sampled candidate from a capable LLM often satisfies the verifier. In our run this interaction produced the near-ceiling baseline, illustrating that generator design choices (ambiguity, under-specification, and difficulty-sampling) materially influence the observable marginal benefit of any repair stage.

Semantics, sampling, and estimand trade-offs. The execution semantics (`shared_initial_candidate`) intentionally reduced per-intent sampling variance by using the same initial candidate across conditions; this made the guarded-baseline contrast a pure marginal-repair estimand. Alternative semantics— independent per-condition sampling or ensembles of initial draws—estimate different operational questions: independent sampling measures expected verifier pass rate when each pipeline draws independently from the model distribution, while multi-draw or multi-repair budgets measure how resampling or repeated repairs trade cost for improved correctness. Independent sampling or larger repair budgets typically increase the chance of observing discordant pairs ($n_{10} + n_{01}$) and therefore can increase power to detect smaller absolute differences, but at the cost of additional model calls and latency. Practitioners should select sampling semantics that match deployment constraints (whether systems will resample or must act on a single first-pass candidate) and incorporate model-call cost and repair-latency budgets into experimental power planning.

Repair budgets, cost, and diminishing returns. The provider accounting from our run (`model_calls_used` = 151; `repair-stage` invocations = 1) quantifies the operational cost of the guarded design under the one-repair budget: repairs were rarely invoked when the baseline was near-ceiling, so marginal cost per observed success-gain was effectively high. In scenarios with lower baseline competence, repair budgets exhibit diminishing returns: initial repairs convert many failures, but additional repairs yield progressively fewer conversions. Quantifying diminishing returns requires multi-arm experiments that vary allowed repair counts and measure per-repair marginal gain versus model-call cost; our artifacts supply seeds and code to run such arms reproducibly.

Operational affordances beyond pass/fail. Deterministic guarded workflows provide benefits not captured by a single binary endpoint. The validator produces `normalized_configuration` strings, `parsed_command_count`, `required_command_count`, and explicit `violation_codes` that op-

erators can audit; these artifacts accelerate triage and human review even when verifier-pass differences are numerically small. For example, representative validator traces in the archive show concrete violation_code semantics (missing step, order-violation) that would guide automated or human repair policies. Thus, guarded pipelines can improve operational observability, reduce human triage time, and support auditable change records independent of marginal verifier-pass improvements on constrained corpora.

Threats to validity and residual risks. Internal validity: adapter-enforced call budgets and the provider audit mitigate cross-condition leakage, and mapping unit-tests reduced parse/translation failures. Construct validity: deterministic validator pass/fail is a proxy for semantic correctness and depends on mapping code and verifier coverage; although mapping unit-tests passed and no parse failures occurred, any uncovered verifier-coverage gap would bias semantic-correctness measurement. External validity: experiments used a single declared model (gpt-5-mini) and synthetic tasks on small topologies (4–32 nodes); results should not be extrapolated to larger, heterogeneous networks or other models. Security and adversarial robustness: this confirmatory trial did not evaluate adversarial prompt-injection; prior work indicates claim-to-action attacks remain an independent concern and defensive evaluation requires separate adversary-driven experiments. Conclusion validity: the mismatch between preregistered detectable-effect assumptions and the observed baseline underscores the importance of pilot calibration to preserve inferential validity.

Practical experimental prescriptions. Based on artifact-grounded outcomes we recommend: (1) a baseline-calibration pilot (20–40 tasks) to estimate baseline_success_rate and discordant-pair proportions prior to final N selection; (2) explicit preregistration of generation semantics and repair budgets; (3) multi-arm designs that vary repair budgets and sampling semantics to estimate cost–benefit frontiers; (4) inclusion of provider-call accounting in power and cost planning; and (5) staged scaling of verifier coverage and topology size with deterministic unit-tests for mapping code. The archived execution manifest and representative tasks provide concrete seeds and runnable code to implement these prescriptions reproducibly.

VI. LIMITATIONS

- Single-model constraint: experiments used only gpt-5-mini; results do not generalize across models or sizes.
- Synthetic corpus and scale: tasks were programmatically generated and limited to verifier-capable toy-to-small topologies (4–32 nodes); external validity to production WANs and heterogeneous vendor CLIs is untested.
- Shared_initial_candidate semantics and execution contract limited repair budget to at most one guarded repair call per intent; different generation semantics or multiple-repair budgets may yield different outcomes.
- Ceiling effect and preregistration mismatch: baseline correctness was far higher than the pilot assumptions

used for power calculations (planned detectable effect = 15 percentage points), reducing the trial’s ability to demonstrate benefit and illustrating sensitivity to baseline calibration.

- Verifier coverage and specification translation: the study measures deterministic validator pass/fail on the preregistered invariants but does not claim safety guarantees beyond the deterministic validator’s modeled properties.
- Security and adversarial robustness: the experiment did not evaluate adversarial prompt-injection or adversary-driven intents; separate targeted studies are required to assess claim-to-action vulnerabilities in agentic NetOps workflows.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory trial comparing baseline free-text prompting with constrained-template prompting plus a single verifier-guided repair under shared_initial_candidate semantics. On a deterministic synthetic corpus with gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline increased verifier pass rate from 149/150 to 150/150 (≈ 0.67 pp), a numerically tiny and statistically non-significant difference (exact McNemar $p = 1.0$; 95% bootstrap CI = [0.00, 0.02]). The principal scientific lesson is methodological: preregistered NetOps trials should include baseline calibration to avoid ceiling effects and preserve statistical power. Technically, our artifacts bound the marginal benefit of a one-repair guarded pipeline under shared_initial_candidate semantics; evaluating independent-sampling semantics, larger repair budgets, model diversity, and production-scale topologies remains important future work.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Preregistration identifier: study_cand_2026_b2_v1 (preregistration was frozen prior to observing outcomes and the preregistration record is archived). Declared model/tooling: declared model = gpt-5-mini (provider record: openai_responses); execution adapter family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5 (validator_version = 5.0). Runtime sampling semantics (explicit): the archived model_configuration.json records temperature = null (unset); because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions. Autonomy and human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the autonomous system performed literature review, study selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No human

manually edited technical content, analyses, figures, captions, or references after the autonomy lock. Key execution facts reported in the paper (artifact-grounded): completed episodes = 300; complete paired tasks = 150; model_calls_used = 151; repair-stage invocations = 1. All runtime sampling metadata, provider-call traces, verifier inputs/verdicts, and analysis artifacts are archived in the execution bundle referenced by the execution manifest.

REFERENCES

- [1] <https://doi.org/10.1002/nem.2313>
- [2] <https://doi.org/10.1145/3786583.3786898>
- [3] <https://arxiv.org/abs/2604.22513>
- [4] <https://doi.org/10.1145/3605764.3623985>
- [5] <https://doi.org/10.1145/3656454>
- [6] <https://doi.org/10.1002/widm.70111>